Submission Worksheet

Submission Data

Course: IT202-008-S2026
Assignment: IT202 Module 3 HTML, CSS, JS Challenges
Student: Stephane E. (se283)
Status: Submitted | **Worksheet Progress:** 100+%
Potential Grade: 10.99/10.00 (109.90%)
Received Grade: 0.00/10.00 (0.00%)
Started: 3/3/2026 1:17:57 PM
Updated: 3/9/2026 11:46:38 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-module-3-html-css-js-challenges/grading/se283
View Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-module-3-html-css-js-challenges/view/se283

Instructions

- Overview Link: https://youtu.be/Dyl6dg1Xybo (older but should still help)

1. Ensure you read all instructions and objectives before starting.
2. Create a new branch from `qa` called `M3-Homework`
    1. `git checkout qa` (ensure proper starting branch)
    2. `git pull origin qa` (ensure history is up to date)
    3. `git checkout -b M3-Homework` (create and switch to branch)
3. Copy the template code from here: GitHub Repository - M3 Homework
    - It includes Challenges 1-3, `util.js`, and `styles.css`. Put all into an `M3` folder or similar inside your `public_html`
    - Immediately record to history
        - `git add public_html`
        - `git commit -m "adding M3 HW baseline files"`
        - `git push origin M3-Homework`
        - Create a Pull Request from `M3-Homework` to `qa` and keep it open
4. Fill out the below worksheet
    - Each Problem requires the following as you work
        - Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
        - Update ucid in header tag
        - Code solution (add/commit periodically as needed) (style and/or script tags)
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin M3-Homework`
    4. On Github merge the pull request from `M3-Homework` to `qa`
    5. On Github create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod`

deploy to make the render prod links work)

7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout qa`
    2. `git pull origin qa`

Section #1: (3 pts.) Challenge 1 - Fixed Header, Content, Footer

Progress: 100%

≡ Task #1 (3 pts.) - Edit the `style` and `script` tags to solve the challenge requirements

Progress: 100%

Details:

- Only make edits where noted via provided comments
- Update your ucid in the header tag
- #1 The header and footer should remain FIXED in place (top and bottom of page respectively)
- #2 The content area should SCROLL independently (nothing should be pushed off screen and the browser WINDOW scrollbar shouldn't appear)
- #3 The entire page should always take up the full viewport height
- #4 The borders around header,main,footer should remain intact and visible, this will help show that the challenges were solved correctly
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:
Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (visit the proper file on render.com qa after a manual deploy) 1. Ensure url is visible in the browser's address bar

Snippet of relevant code showing solution (with ucid/date comment)

Full output of executing the program (visit the proper file on render.com qa after a manual deploy) 1. Ensure url is visible in

Saved: 3/9/2026 11:38:47 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in `.html`,, zero credit if it does not)
- Direct link to the file on Render.com Prod (Grab the qa url you're testing, paste it, change "-qa" to "-prod"), zero credit if

URL #1
https://github.com/se283/se283-it202-[redacted]/Homework/public_html/M3/challenge1.html

URL
https://github.com/se283/se283-i

URL #2
https://se283-it202-008-prod.onrender.com/M3/challenge1.html

URL
https://se283-it202-008-prod.onre

Saved: 3/9/2026 11:38:47 PM

Part 3:

Progress: 100%

Details:
Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

To reach the result, the code turns the entire webpage into a single vertical stack by setting the body to "display: flex" and "flex-direction: column". By forcing the body height to exactly 100vh, the layout is locked to the size of the screen, and "overflow: hidden" is used to disable the browser's main scrollbar. While the nav, header, and footer are told to stay their natural size, the main

section is given "flex-grow: 1" to stretch and fill all the remaining space in the middle. Finally, "overflow-y: auto" is applied specifically to that main section, which allows only the content inside the green box to scroll while keeping all borders perfectly aligned and visible.

Saved: 3/9/2026 11:38:47 PM

Section #2: (3 pts.) Challenge 2 - Header, Content, And Sidebars

Progress: 100%

Task #1 (3 pts.) - Edit the `style` and `script` tags to solve the challenge requirements

Progress: 100%

Details:

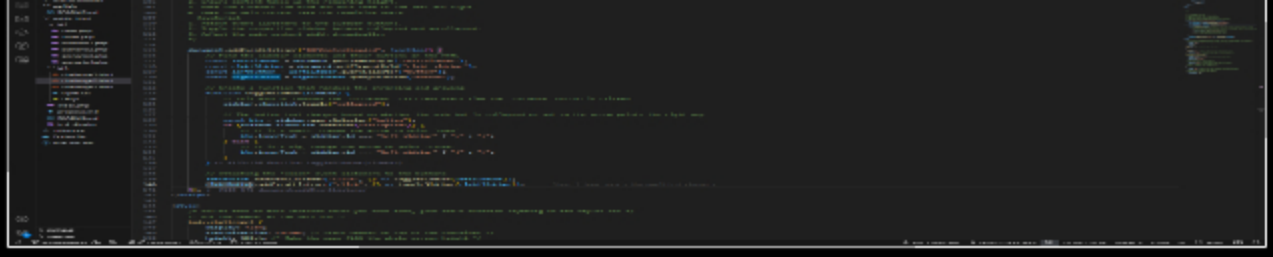
- Only make edits where noted via provided comments
- Update your ucid in the header tag
- Using CSS, adjust the layout per the following
  - #1: Header is at the top
  - #2: Content takes up the rest of the height
  - #3: Both sidebars docked to the respective side and take up 15% width; Content area should utilize the remaining width
- Using JavaScript complete the following
  - #4: Attach the appropriate event listener to the buttons
  - #5: Individual toggle the respective panel between collapsed and uncollapsed (Don't lose the button in the process)
  - #6: The content should adjust based on the status of the respective sidebars (i.e., take up more left, right, or both space)
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:
Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (visit the proper file on render.com qa after a manual deploy) 1. Ensure url is visible in the browser's address bar



Snippet of relevant code showing solution (with ucid/date comment) - JS

Full output of executing the program (visit the proper file on render.com qa after a manual deploy) 1. Ensure url is visible in

Snippet of relevant code showing solution (with ucid/date comment) - CSS

Saved: 3/9/2026 11:39:01 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in `.html`, , zero credit if it does not)
- Direct link to the file on Render.com Prod (Grab the qa url you're testing, paste it, change "-qa" to "-prod"), zero credit if

URL #1

https://github.com/se283/se283-it202-[redacted]/Homework/public_html/M3/challenge2.html

URL

https://github.com/se283/se283-i

URL #2

URL

https://se283-it202-008-prod.onre

https://se283-fl202-008-prod.onrender.com/M3/challenge2.html

Saved: 3/9/2026 11:39:01 PM

Part 3:

Progress: 100%

Details:
Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

The solution uses a Flexbox layout to divide the screen into a vertical stack where the header stays at the top and the container fills the remaining height. Inside that container, the sidebars are set to a initial width of 15%, while the main content is given a "flex: 1" property, which acts like a rubber band to automatically stretch and fill any leftover horizontal space. When a user clicks a sidebar button, a JavaScript event listener toggles a ".collapsed" class on that specific sidebar. This class overrides the width to a small fixed value and hides the list items, causing the flexible main content area to instantly expand and occupy the newly available room.

Saved: 3/9/2026 11:39:01 PM

Section #3: (3 pts.) Challenge 3 - Carousel Layout With Swiping

Progress: 100%

Task #1 (3 pts.) - Edit the `style` and `script` tags to solve the challenge requirements

Progress: 100%

Details:

- Only make edits where noted via provided comments
- Update your ucid in the header tag
- Using CSS, adjust the layout per the following
  - #1 Header should be at the top
  - #2 Carousel should take up full width
  - #3 Buttons should be centered
  - #4 Carousel panels should fill the full height of the carousel
  - #5 Carousel content should be centered (vertical and horizontal)
- Using JavaScript complete the following
  - #4 Attach appropriate event listeners to each button
  - #5 Cycle through each panel, showing only 1 at a time
  - #6 Ensure that the panels loop when reaching the last or first one
- Add code to solve the problem (add/commit as needed)

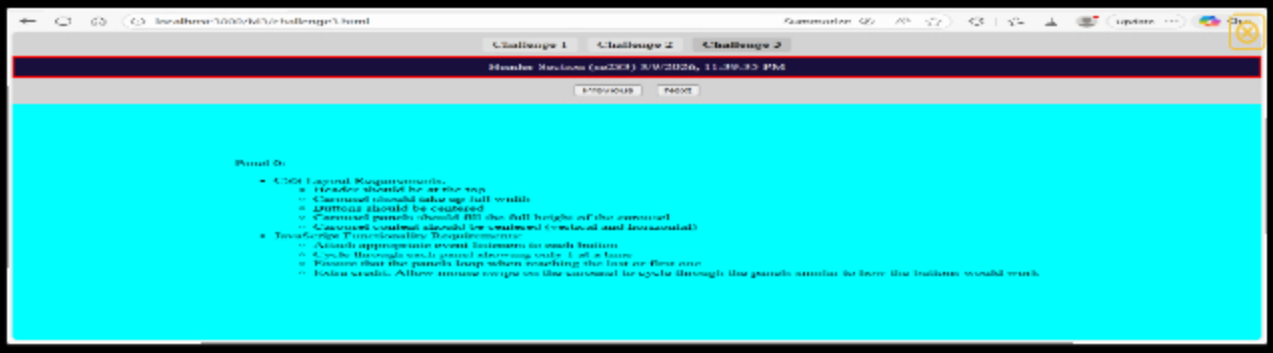
Part 1:

Progress: 100%

Details:
Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (visit the proper file on render.com qa after a manual deploy) 1. Ensure url is visible in the browser's address bar

Snippet of relevant code showing solution (with ucid/date comment) - JS



Full output of executing the program (visit the proper file on render.com qa after a manual deploy) 1. Ensure url is visible in

Snippet of relevant code showing solution (with ucid/date comment) - CSS

Saved: 3/9/2026 11:43:41 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in `.html`, , zero credit if it does not)
- Direct link to the file on Render.com Prod (Grab the qa url you're testing, paste it, change "-qa" to "-prod"), zero credit if

URL #1
https://github.com/se283/se283-it202-[illegible]/Homework/public_html/M3/challenge3.html

URL
https://github.com/se283/se283-i

URL #2
https://se283-it202-008-prod.onrender.com/M3/challenge3.html

URL
https://se283-it202-008-prod.onre

Saved: 3/9/2026 11:43:41 PM

Part 3:

Progress: 100%

Details:
Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

The solution uses a fixed header at the top and a centered button bar at the bottom. The carousel acts like a window that shows only one panel at a time while hiding the others. Inside this window, all panels sit side-by-side in one long row. JavaScript tracks which panel is active using a number. When a button is clicked, the code changes that number and slides the row left or right by 100% to show the next slide. If the number goes past the last slide, it loops back to the start.

Saved: 3/9/2026 11:43:41 PM

Task #2 (+ 0.99 pts.) - Extra Credit - Challenge 7

Progress: 100%

Details:

- Allow mouse swipe on the carousel to cycle through the panels, similar to how the buttons would work

Part 1:

Progress: 100%

Details:

1. Snippet of relevant code showing solution (with ucid/date comment)

Snippet of relevant code showing solution (with ucid/date comment)

Saved: 3/9/2026 11:43:55 PM

Part 2:

Progress: 100%

Details:
Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

The swipe feature works by tracking the movement of the mouse across the carousel area. When a click starts, the code records the horizontal starting position. When the mouse button is released, the code captures the final horizontal position and calculates the distance moved. If the drag distance is greater than 50 pixels, the script decides whether to move to the next or previous panel based on the direction of the swipe. Once the direction is determined, the code updates the panel index and slides the container to show the new slide.

Saved: 3/9/2026 11:43:55 PM

Section #4: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history

From the Commits tab of the Pull Request screenshot the commit history

Saved: 3/9/2026 11:11:52 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/se283/se283-it202-008-2026/

URL
https://github.com/se283/se283-i

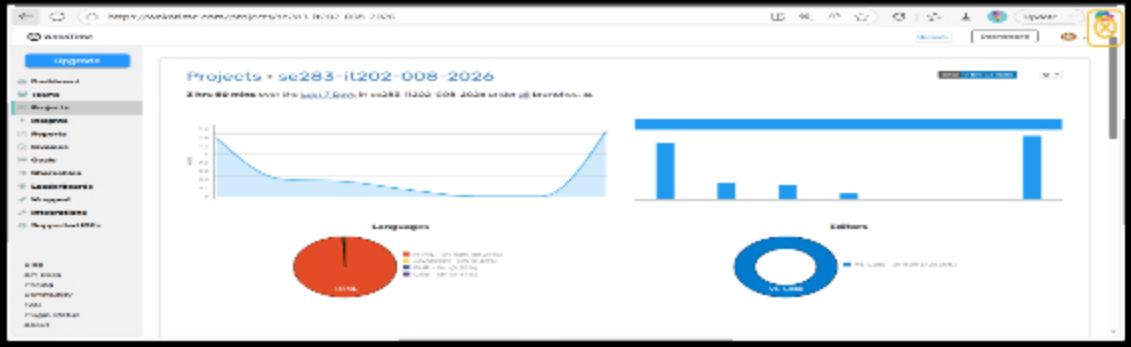
Saved: 3/9/2026 11:11:52 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Capture the overall time at the top that includes the repository name

Capture the individual time at the bottom that includes the file time

Saved: 3/9/2026 11:06:34 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

One thing I learn from this assignment is the Flexbox layout model in CSS. Flexbox comes with properties that I've learned about, such as 'flex-direction', 'flex-shrink', and 'flex-grow'.

Saved: 3/3/2026 7:22:47 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignment was using CSS Flexbox to organize the page layout. By telling the browser to treat the page like a flexible container, you could

easily stack the header, middle content, and footer in a straight line. Setting the middle section to "grow" allowed it to automatically fill all the empty space on the screen, while a simple "overflow" setting made that area scrollable without moving the header or footer. This modern approach replaced older, more complicated methods, making it very straightforward to keep everything perfectly aligned and sized to fit the browser window.

Saved: 3/9/2026 11:46:38 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was dealing with the CSS and JavaScript code as I tried to balance design/layout and interactivity, making sure the page appeared the way it was supposed to appear while also making it properly do what it was meant to do, all without messing up either one of them as I work on the other.

Saved: 3/9/2026 11:01:26 PM